FC: Adaptive Atomic Commit via Failure Detection

Hexiang Pan*, Quang-Trung Ta*, Meihui Zhang‡, Zhanhao Zhao*,
Yeow Meng Chee*, Gang Chen†, Beng Chin Ooi*
*National University of Singapore †Zhejiang University ‡Beijing Institute of Technology
panh@u.nus.edu, ymchee@nus.edu.sg, *cg@zju.edu.cn* *meihui_zhang@bit.edu.cn*
{taqt, zhzhao, ooibc}@comp.nus.edu.sg

***Abstract*—Atomic commit protocols (ACPs) are crucial for ensuring transaction atomicity in distributed transaction processing. However, existing ACPs, designed specifically for fixed failure conditions, cannot work efficiently in modern environments, where failures such as node crashes and connection delays can happen anytime due to the use of commodity nodes and networks.**

In this paper, we propose FC, a novel and practical ACP that can adapt to changes in failure conditions. In essence, FC includes three dedicated protocols, which are specifically designed for three different failure conditions: (i) failure-free: no failure occurs, (ii) crash-failure: nodes might crash but there is no delayed connection, or (iii) network-failure: both crashed nodes and delayed connection can occur. During its operation, FC can monitor if any failure occurs and dynamically switch to the most suitable protocol, using a protocol selector, whose parameters are fine-tuned by reinforcement learning. Consequently, FC improves transaction performance and robustly ensures fault tolerance when crash failures and network failures occur. We conduct extensive experiments to evaluate FC with both YCSB and TPC-C benchmarks. The experimental results show that FC achieves up to 2.88x higher throughput and 3.76x lower latency than state-of-the-art ACPs, and its sustainable performance when integrated with two popular databases, namely MongoDB and PostgreSQL.

***Index Terms*—Atomic Commit, Distributed Transactions**

I. Introduction

Database systems that support critical applications require all transactions to guarantee the ACID properties (atomicity, consistency, isolation, durability) [1]. Among them, the atomicity property, which mandates a transaction must happen in its entirety, is difficult to guarantee in sharded or distributed databases. These databases typically partition data across multiple disjoint nodes for horizontal scalability. Consequently, for a distributed transaction that spans multiple nodes, achieving atomicity necessitates all these nodes to agree on a common status (either commit or abort) of the transaction.

Atomic commit protocols (ACPs) are proposed to ensure atomicity in distributed transactions. However, existing ACPs tend to exhibit sub-optimal performance, whether in the absence of failures, during failures, or in both scenarios. Two-phase commit (2PC) [2], the most prevalent ACP, begins by a *prepare* phase where a coordinator collects votes from all involved nodes (a.k.a. participants) on whether to commit the transaction. This phase is then followed by a *commit* phase in which the coordinator decides the transaction's final status and dispatches this decision to all participants for agreement. While 2PC effectively ensures atomicity, it suffers from the blocking problem when failures occur [2], [3]. Specifically, if the coordinator crashes after collecting all the votes, the transaction may be blocked due to the unknown decision to commit or abort. To alleviate this blocking problem, three-phase commit (3PC) [4], [5] is proposed. 3PC adds a *pre-commit* phase to the 2PC process, allowing participants to independently determine the transaction's outcome when the coordinator fails, thus avoiding the blocking of the transaction on alive participant nodes. However, this additional phase introduces an extra message round trip for all transactions, significantly degrading system performance when no failure occurs. On another line of research, replication-enhanced protocols [6]–[9] address the blocking issue by replicating transaction logs. However, their use of replication incurs additional network and computation overheads due to replica synchronization.

With the prevalent use of commodity compute nodes and networks in modern distributed databases, node crashes or network failures can happen anytime [10]–[12]. Consequently, the database may operate under varying failure conditions over time, such as failure-free conditions and crash-failure conditions. Unfortunately, existing protocols fail to perform well under changing failure conditions due to a limitation – they do not consider the changes in failure conditions and use a specific, fixed failure-handling method. More specifically, each protocol adopts a failure-handling method that performs well in a specific failure condition. If the actual failure condition does not match the protocol's expectation, the resulting overhead from the chosen failure handling method may not justify the optimization they provide. For example, 2PC performs well in failure-free conditions but may cause the aforementioned blocking problem when the coordinator crashes, while 3PC is designed to manage failures yet at the cost of reduced performance in failure-free conditions. We must emphasize that a good ACP should fulfill two requirements: 1) it should deliver high performance when no failures occur; 2) it should tolerate failures without causing transaction blocking, thus maintaining system availability and efficiency. In light of this, we propose to design an adaptive ACP that can identify the current failure condition and adjust its execution accordingly to ensure efficient and fault-tolerant atomic commit.

In this paper, we introduce FC, the first **F**ailure-aware adaptive atomic **C**ommit protocol. FC is able to seamlessly switch between three dedicated protocols (FC_{FF}, FC_{CF}, and FC_{NF}) specifically optimized for three failure conditions: (i) failure-free (FF): no failure occurs; (ii) crash-failure (CF):

2024 IEEE 40th International Conference on Data Engineering (ICDE) | 979-8-3503-1715-2/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICDE60146.2024.00162

2375-026X/24/$31.00 ©2024 IEEE
DOI 10.1109/ICDE60146.2024.00162

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO CEARA. Downloaded on July 29,2025 at 15:47:08 UTC from IEEE Xplore. Restrictions apply.

nodes might crash but there is no delayed connection, or (iii) network-failure (NF): both crashed nodes and delayed connection can occur. For each transaction, FC determines the most suitable dedicated protocol according to the current failure condition. However, designing FC to achieve high performance requires addressing two challenges: First, each dedicated protocol should be precisely fine-tuned to operate efficiently under its specific failure condition. Second, FC must accurately detect and respond to changes in failure conditions, ensuring the most suitable protocol is selected.

We carefully design FC_{FF}, FC_{CF}, and FC_{NF} to ensure each achieves better performance under its specific failure conditions. ACP leverages the coordinator to communicate with participants over the network to reach an agreement on the transaction's final state. This coordination overhead increases the likelihood of blocking concurrent transactions accessing the same records [13]–[15]. We therefore optimize each dedicated protocol to reduce this overhead under its specified failure conditions to improve performance. For FC_{FF}, designed for failure-free environments, we employ each participant to independently collect all the votes from other participants and commit without requiring the coordinator's intervention. As a result, a transaction in FC_{FF} commits at each participant with 1 message delay (needed for collecting votes), while existing ACPs, such as 2PC, typically require each participant to block transactions for ≥ 2 message delays (one for sending votes to the coordinator and another for awaiting the coordinator's decision). FC_{CF} is specifically designed to efficiently handle crash-failure conditions while ensuring non-blocking processing. To do this, after requiring each participant to collect all votes, FC_{CF} involves the coordinator to make the final decision and directs each participant to act accordingly based on this decision. Consequently, if a participant fails, FC_{CF} enables other participants to independently decide to abort the transaction after 1 message delay, while all existing ACPs require the transaction to be blocked for ≥ 2 message delays before being aborted. We regard FC_{NF} as any ACP that can tolerate both crash and network failures, e.g., 3PC, thus ensuring FC is practical to optimize almost all ACPs.

We propose a learning-enhanced protocol selector to ensure FC can efficiently adapt to the appropriate protocol under varying failure conditions. We first design a rule-based failure detection algorithm to accurately identify the current failure conditions. By analyzing messages related to votes and decisions during protocol execution, this algorithm detects failures without incurring any additional overhead. Given the fact that protocols designed for more severe failure conditions (e.g., network-failure conditions) inherently ensure better failure tolerance compared to those designed for less severe failure conditions (e.g., failure-free conditions), we then establish a protocol transition model: immediately upgrade the protocol in response to corresponding failures, and cautiously downgrade it after recovering from such failures. For example, FC_{FF} is upgraded to FC_{CF} upon detecting a crashed node, but downgraded to FC_{FF} after observing continuous successful executions without crash failures. Given the predictability of failure recurrence in modern applications [16], [17], we equip the protocol selector with reinforcement learning (RL) to fine-tune its parameters for better performance. By predicting future failures, this RL-based optimizer avoids repetitive protocol upgrades and downgrades if failures occur recurrently.

In summary, we make the following contributions:

- We propose FC, the first failure-aware adaptive commit protocol that can self-adjust its execution by using different dedicated protocols to handle different failure conditions.
- We optimize the dedicated protocols FC_{FF} and FC_{CF}, to ensure they can be more efficient than existing ACPs in the specific failure conditions.
- We design a failure-aware protocol selector to effectively select suitable dedicated protocols for FC. We apply reinforcement learning to optimize the transition model of FC to tolerate repetitive failures.
- We implement FC on a distributed transactional key-value store prototype based on Google Percolator [18], and two real-world databases, MongoDB, and PostgreSQL to evaluate the FC's performance and applicability.
- We conduct extensive evaluations of FC with state-of-the-art ACPs with YCSB and TPC-C benchmarks. Our experiments show that FC achieves 2.88x higher throughput than other ACPs, and sustainable performance gain in real-world databases.

The rest of this paper is organized as follows. Section II introduces the background of ACPs. Section III, IV present the design of our FC protocol: its system model, protocols, and transaction processing system. Section V introduces the protocol selector, and shows how FC combines three protocols with it. Section VI includes the failure handling and correctness proof of FC. Section VII exhibits our performance evaluation of FC and existing ACPs. We then review related works in Section VIII, and finally conclude in Section IX.

II. Background

A. Atomic Commit Protocol and its Blocking Problem

Database systems utilize atomic commit protocol (ACP) to enforce consistent and valid decisions for a distributed transaction across multiple nodes. The decision is always based on the "votes" from nodes indicating their local checks for the transaction. A node votes No if it wants to abort a transaction due to local invalid executions that violate ACID properties, process crashes, etc. Otherwise, the node votes Yes to inform other nodes it is ready to commit. Once enough information is collected, each node will decide to commit or abort locally. Existing research [3], [19] defines the requirements for ACP as safety and liveness. For safety, an ACP must ensure that all nodes make the same decision (*agreement*), and $Commit$ decision is only made when all nodes vote Yes (*commit validity*). For liveness, an ACP is expected to decide $Commit$ if all nodes vote Yes without failures (*abort validity*), and avoid blocking alive nodes when failures occur (*termination*). Among these properties, safety is always ensured for protocol correctness, but liveness (*termination*) can be compromised in protocols such as 2PC.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO CEARA. Downloaded on July 29,2025 at 15:47:08 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Comparison of existing ACPs' failure handling and normal processing efficiency.

Protocol	Failure Handling	Normal Processing
2PC [2]	Blocking	Baseline
3PC, PAC, EC [3], [4], [9]	Abort w/o block	w/ extra messages
PaxosCommit, TAPIR, GPAC, MDCC [6]–[9]	Commit via leader transfer	w/ extra messages & replication overhead

One critical problem of ACP is that it introduces significant delays to distributed transactions, as highlighted in prior research [13]–[15]. To ensure a consistent decision across multiple participant nodes, a participant voting Yes must gather enough information from other participants before safely persisting or reverting its changes to local records. It causes the block of all concurrent transactions that access these undecided records. This blocking effect intensifies in geo-distributed systems with large network delays and skewed workloads that entail hotspots. It motivates us to further reduce ACP-introduced transaction delays by making ACP adaptive.

B. The Failure Model and Safe Assumptions

Real systems often face different kinds of failures, such as process crashes, message losses, and Byzantine faults [20], [21]. It is therefore not easy to consider all possible failures when designing systems. Instead, the designer often proclaims a set of failures as a basis. In this work, we adopt the failure model used by existing ACPs (such as 2PC, 3PC and other protocols [2]–[4], [22]–[24]) that crash failure is considered and point-to-point network communications never fail but could be delayed. We also employ their assumption that the receiver node can reliably detect the crash failure of the sender node by a timeout, which makes it possible to differentiate between crashed nodes and slow nodes [21], [25]. We call this timeout as *crash timeout*, i.e., the timeout which is used to detect crash failures. In situations where the protocol functions under conditions with unbounded network delays, the *crash timeout* is infinite. Additionally, considering the stability of modern networks, we follow [26] to use a measured *network timeout* to accommodate non-delayed messages that expose low variability of delays. In our implementation, we consider there is a network failure if we fail to receive a message within the network timeout W_{C_i}, and a crash failure of the sender node if we fail to receive a message within the crash timeout.

Although using the same failure model, existing ACPs adopt varying failure handling techniques. As summarized in Table I, each protocol uses a specific, fixed failure handling method. These methods significantly impact performance during both the normal processing and the failure handling. In particular, protocols designed for efficient failure handling, such as those enhanced by replication, often incur additional overhead during their normal processing. This results in reduced performance compared to protocols with lower failure tolerance, such as 2PC, which however recover/terminate less swiftly and effectively when failures occur. Given the dynamic nature of failure conditions in databases, as discussed in Section I, relying on a static failure-handling approach may lead to suboptimal performance under changing circumstances. This observation motivates us to optimize a commit protocol for each specific failure condition and unify them with an adaptive execution framework. Based on the aforementioned failure model, we identified three critical environments:

(i) Failure-free environments: the environments that rarely contain any crashed nodes or delayed messages.
(ii) Crash-failure environments: the environments in which nodes might crash but messages are rarely delayed.
(iii) Network-failure environments: the environments where the crashed nodes and delayed messages can both occur.

The above classification enables us to implement two novel optimizations for failure-free and crash-failure environments. First, in a failure-free environment, an ACP does not need to consider crash failures and thus allows participants to make decisions independently for higher efficiency. Second, in both failure-free and crash-failure environments where all messages arrive on time, a node can try to synchronize with others by broadcasting a message and waiting for a short network timeout [26] (one message delay), instead of sending round-trip messages to synchronize (two message delays) like other ACPs [2]–[4], [22]. Moreover, in crash-failure environments, since all messages arrive within the network timeout, the message receiver can quickly assert the crash failure of the sender and independently abort affected transactions. Both optimizations mitigate the commit protocol's exacerbation of contention among concurrent transactions.

III. System Overview

A. Architecture

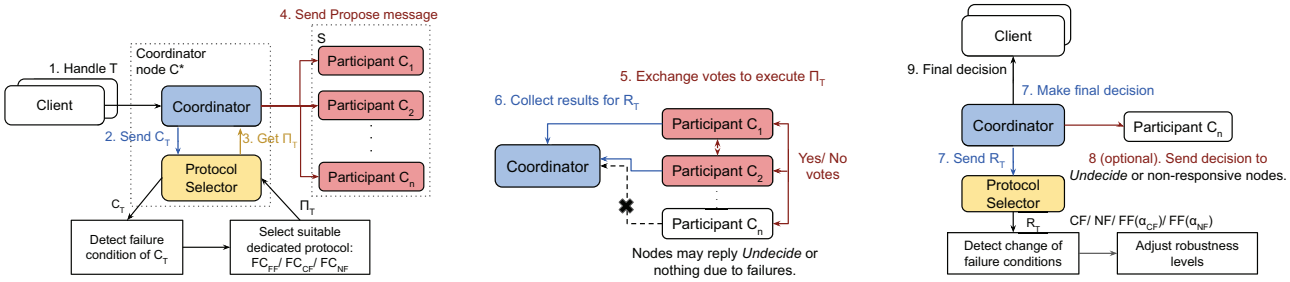
We design FC using a typical transactional system architecture, where one node is designated as the coordinator, while the other nodes serve as participants. Its system model is depicted in Figure 1a, where C^* is the coordinator and $\mathcal{S} = \{C_1, ..., C_n\}$ are the participants. Each participant manages a local data store, while the coordinator serves as a transaction coordinator which accepts and manages transactions from clients. A transaction T may read or update data stored in multiple nodes that participate in T's execution. We denote these participant nodes as $\mathcal{C}_T$ ($\mathcal{C}_T \subseteq \mathcal{S}$). We also equip the coordinator node C^* with a component called protocol selector, which is responsible for determining the suitable protocol Π_T to execute T.

We summarize our notations in Table II. In our system model, we follow existing works [2]–[4], [27] to assume *reliable* but *delayable* connections between nodes. Like other ACPs, FC enables the use of the coordinator as a participant and the use of multiple coordinator nodes. The former is a special case of our current system model, in which connections between a participant and the coordinator happen in the same node. The latter is supported since conflicts between concurrent transactions are resolved at participant nodes.

B. Transaction Processing

We now describe how FC processes transactions. It consists of three dedicated protocols, namely FC_{FF}, FC_{CF}, FC_{NF}, specifically designed for the failure-free, the crash-failure,

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO CEARA. Downloaded on July 29,2025 at 15:47:08 UTC from IEEE Xplore. Restrictions apply.



(a) Transaction initiation (b) Propose phase (c) Validate phase

Fig. 1: System architecture and transaction processing workflow of FC.

TABLE II: Symbols and descriptions

$\mathcal{S}$	the system of n participant nodes $C_1, ..., C_n$
C^*	the coordinator node
T	the transaction sent to C^*
$\mathcal{C}_T$	the participant nodes contributing to the commit of T
$\mathcal{R}_T$	the tentative execution results of $\mathcal{C}_T$ for T
Π_T	the protocol selected for T

and the network-failure environment, respectively. Given a transaction to be processed, the coordinator C^* first selects the most suitable protocol, denoted as Π_T, based upon the current failure conditions of the participant nodes $\mathcal{C}_T$ that execute T (steps 1-4 Figure 1a).Then, FC enters the *propose* phase (Figure 1b). In this phase, all participants in $\mathcal{C}_T$ will jointly execute Π_T, exchanging votes with each other and making local decisions when collecting sufficient votes to decide $Commit$ or $Abort$ (step 5). Afterward, they send their execution results to C^* (step 6). These results, denoted as $\mathcal{R}_T$, include the participants' votes and decisions. If a participant fails to decide in this process (steps 5-6), it is called *undecided participant* and will reply $Undecide$ instead of $Commit$ or $Abort$. Finally, FC will enter the *validate* phase (Figure 1c). In this phase, the coordinator C^* decides to commit/abort the transaction based on collected results and sends that decision to all undecided participants (steps 7-9). Since we consider crash and network failures, some participants in $\mathcal{C}_T$ might crash, or their replies might not reach the coordinator within the timeout due to delay. $\mathcal{R}_T$ does not include their results.

As discussed in Section I and II, selecting a suitable commit protocol for each transaction is critical for system performance. Hence, we equip FC with protocol selector (Figure 2), which tracks the failure condition of each participant. More specifically, we first represent three failure conditions in Section II-B as three robustness levels: failure-free (L_{FF}), crash failure (L_{CF}), and network failure (L_{NF}). During a transaction execution, protocol selector analyzes the results reported by participants to detect if any failure occurs. If an unexpected failure occurs in a participant node (e.g., network failure or crash failure), it will upgrade this node's robustness level to a more stringent one. Otherwise, protocol selector will downgrade robustness levels. Failures of the coordinator do not affect FC's normal processing. We detail the handle of coordinator failures in Section VI-A.

In FC, unexpected failures may intensify the blocking of half-executed transactions (happening when running the dedicated protocol FC_{FF}) or cause committable transactions to abort (happening when running the dedicated protocol FC_{CF}). Thus, to mitigate the occurrence of unexpected failures in rapidly changing environments, we take a cautious approach in robustness downgrading transitions. During a transaction execution, if no failure is detected, protocol selector will record the total number of consecutive transactions that have been executed successfully in each node. If this number reaches a threshold representing the environment's stability, FC will downgrade the node's robustness level to a more lenient level. Later in Section V, we shall describe how to use reinforcement learning to fine-tune this threshold so that FC can adapt well to changes in the failure condition.

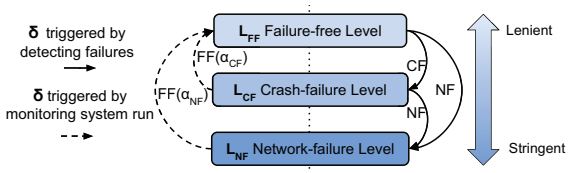


Fig. 2: Protocol selector transition diagram. CF and NF denote executions that experience crash and network failures, respectively. FF(α) signifies α consecutive executions without any failures.

IV. Dedicated Protocols for Different Failure Conditions

This section describes the three dedicated protocols FC_{FF}, FC_{CF}, FC_{NF} that our FC protocol uses to operate in three different environments: failure-free, crash-failure, and network-failure. We first present details of each protocol and elaborate their execution paths. Then, we compare them with existing ACPs and discuss their strengths and weaknesses.

A. FC_{FF} for Failure-Free Environment

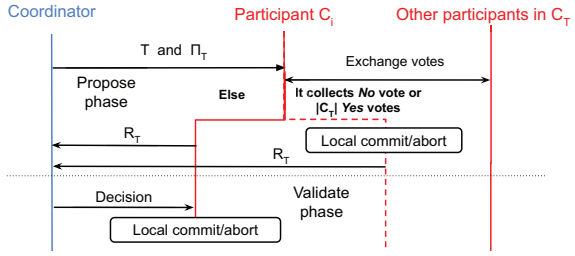


Fig. 3: Timeline of FC_{FF}.

In contrast to existing protocols that assume failures can always happen, FC_{FF} is specifically tailored for environments

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO CEARA. Downloaded on July 29,2025 at 15:47:08 UTC from IEEE Xplore. Restrictions apply.

where failures are either infrequent or non-existent. It prioritizes normal processing efficiency over failure handling efficiency, enabling the prompt initiation of commit decisions on participant nodes to minimize the delay of distributed transactions. We describe the FC_{FF} protocol for the coordinator and the participants in Algorithms 1 and 2, respectively. Its timeline is also illustrated in Figure 3. The coordinator initiates the execution of the protocol on receiving a transaction T from clients. It sends the $Propose$ message to all involved participants $\mathcal{C}_T$ and then waits for their execution results in a network timeout window W_{C^*} (Algorithm 1, lines 1–2).

Algorithm 1: FC_{FF} for the coordinator node C^*.

Data: T, $\mathcal{C}_T$ – the input transaction and its participant nodes.
Result: the final global decision for T.
1 Broadcast $Propose$ messages with (T, C_T) to all $\mathcal{C}_T$
2 $W_{C^*} = max\{\sigma(C^*, C_j) \times r \mid C_j \in \mathcal{C}_T\}$
3 $\mathcal{R}_T \leftarrow$ execution results for T from C_T that arrive in W_{C^*} time.
4 **if** R_T contains $v = Abort$ or $v = Commit$ **then**
5 Broadcast v to nodes in $\mathcal{C}_T$ who have not replied decisions.
6 **return** v
7 **if** $|\mathcal{R}_T| = |\mathcal{C}_T|$ **then**
8 Broadcast $Commit$ to all participants in C_T
9 **return** $Commit$
10 **else** run termination protocol

Algorithm 2: FC_{FF} for each participant node C_i.

Data: T, $\mathcal{C}_T$: the input transaction and its participant nodes.
Result: the final decision of C_i for T.
11 $v \leftarrow$ DetermineLocalVote(T) // the vote of C_i
12 Broadcast v for T to all other nodes in $\mathcal{C}_T$
13 **if** v is No **then**
14 Send back $\langle No, Abort\rangle$ to C^*
15 **return** $Abort$
16 $W_{C_i} = max\{\sigma(C_i, C_j) \times r \mid C_j \in \mathcal{C}_T\}$
17 **if** receive a No for T from a node in $\mathcal{C}_T$ within W_{C_i} time **then**
18 Send back $\langle Yes, Abort\rangle$ to C^*
19 **return** $Abort$
20 **if** receive Yes for T from all nodes in $\mathcal{C}_T$ within W_{C_i} time **then**
21 Send back $\langle Yes, Commit\rangle$ to C^*
22 **return** $Commit$
23 Send back $\langle Yes, Undecide\rangle$ to C^*
24 **if** receive a decision d from C^* within *crash timeout* **then**
25 **return** d
26 **else** run termination protocol

Note that we follow recent studies [26], [28]–[31] to use clocks to measure the network timeout window (W_{C^*} and W_{C_i}), accommodating results or votes sent from failure-free participants. For delay prediction, we adopt the method used by existing works [26], [29], [30], to take the 95th percentile delay measured in the last second as the basic timeout window $\sigma(x, y)$. We also adjust the timeout window, $\sigma(x, y)$, using a network factor r to accommodate diverse network conditions. More specifically, $W_x = max\{\sigma(x, y) \times r | y \in \mathcal{C}_T\}$, where $\sigma(x, y)$ is the expected time for node x to collect the necessary messages from a failure-free node y.

For each participant C_i, upon receiving the $Propose$ message, it first determines and broadcasts its local vote to other participants (Algorithm 2, lines 11, 12). If this vote is No, it will decide to abort the transaction (lines 13–15). Otherwise, it will exchange votes with other participants within the pre-calculated network timeout W_{C_i} (lines 16–26). Within this timeout, the C_i will decide $Abort$ if it receives a No vote (lines 17–19, Figure 3), or decide $Commit$ if it receives Yes votes from all participants (lines 20–22, Figure 3). Otherwise, C_i will remain $Undecide$, send back this result to the coordinator, and wait for the coordinator's decision (lines 23–26, Figure 3). If it can receive a decision before a crash timeout, it will decide accordingly and finish local execution (lines 24–25). If not, the coordinator might crash, and C_i will initiate a termination protocol to gather decisions and votes from other participant nodes in $\mathcal{C}_T$. It then makes a final decision when enough information is collected (line 26).

For the coordinator C^*, after collecting the result set $\mathcal{R}_T$, it will proceed with one of the following three scenarios. Firstly, if $\mathcal{R}_T$ includes a $Commit$ or $Abort$ decision, C^* will broadcast this decision to $Undecide$ participants, and then returns it to clients (Algorithm 1, lines 4–6). Secondly, if $\mathcal{R}_T$ contains results from all participants who also stay $Undecide$, then all these participants vote Yes[1]. Thus, C^* can simply notify a $Commit$ decision to them and return this decision (lines 7–9). Finally, if $\mathcal{R}_T$ does not contain the results of all participants, C^* will not have enough information to make a decision. Then, it will run the termination protocol (line 10). We detail the termination protocol in Section VI-A.

B. FC_{CF} for Crash-Failure Environment

In contrast to conventional protocols that only prioritize liveness in the context of crash failures, FC_{CF} is designed to ensure both liveness and efficiency in crash-failure environments. Specifically, FC_{CF} addresses two major challenges for distributed transactions in crash-failure environments. Firstly, crash failures can hinder the execution of pending distributed transactions, resulting in the blocking of these transactions on alive nodes until crashed nodes recover. Secondly, in the case of a distributed transaction involving crashed nodes, an alive participant voting Yes may be forced to wait for two message delays (one for sending the Yes vote to the coordinator, one for receiving the $Abort$ decision from the coordinator) to abort the transaction locally. Both challenges affect the commit of normal transactions and subsequently lead to performance degradation of the overall system. To address them, FC_{CF} ensures liveness to avoid blocking pending transactions in case of crash failures. Additionally, it incorporates the network timeout window to quickly abort transactions involving crashed nodes, thereby minimizing their impact on committable transactions. We present the FC_{CF} protocol in Algorithms 3, 4 and Figure 4.

On the coordinator side, to meet the aforementioned requirements, FC_{CF} will quickly abort a transaction if it cannot receive results from all participants after the network timeout W_{C^*} (Algorithm 3, lines 10–12). It is also the only difference between the FC_{CF} and FC_{FF} protocols for the coordinator (Algorithm 3 vs. Algorithm 1).

[1]The result $\langle No, Undecide\rangle$ will never occur, because a node voting No will always decide $Abort$ according to the *commit validity* property.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO CEARA. Downloaded on July 29,2025 at 15:47:08 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3: FC_{CF} for the coordinator node C^*.

Data: T, $\mathcal{C}_T$ – the input transaction and its participant nodes.
Result: the final global decision for T.
1 Broadcast $Propose$ messages with $(T, \mathcal{C}_T)$ to all $\mathcal{C}_T$
2 $W_{C^*} = max\{\sigma(C^*, C_j) \times r \mid C_j \in \mathcal{C}_T\}$
3 $\mathcal{R}_T \leftarrow$ execution results for T from $\mathcal{C}_T$ that arrive in W_{C^*}.
4 **if** R_T contains $v = Abort$ **then**
5 Broadcast v to nodes in $\mathcal{C}_T$ who have not replied decisions
6 **return** v
7 **if** $|\mathcal{R}_T| = |\mathcal{C}_T|$ **then**
8 Broadcast $Commit$ to $\mathcal{C}_T$
9 **return** $Commit$
10 **else**
11 Broadcast $Abort$ to $\mathcal{C}_T$
12 **return** $Abort$

Algorithm 4: FC_{CF} for each participant node C_i.

Data: T, $\mathcal{C}_T$: the input transaction and its participant nodes.
Result: the final decision of C_i for T.
13 $v \leftarrow$ DetermineLocalVote(T) // the vote of C_i
14 Broadcast v vote for T to all nodes in $\mathcal{C}_T$
15 **if** v is No **then**
16 Send back $\langle No, Abort\rangle$ to C^*
17 **return** $Abort$
18 $W_{C_i} = max\{\sigma(C_i, C_j) \times r | C_j \in \mathcal{C}_T\}$
19 **if** receive Yes from all $\mathcal{C}_T$ in W_{C_i} **then**
20 Send back $\langle Yes, Undecide\rangle$ to C^*
21 **if** receive a decision d from $C^* \cup \mathcal{C}_T$ in *crash timeout* **then**
22 Broadcast d for T to all nodes in $\mathcal{C}_T$
23 **return** d
24 **else** run termination protocol
25 **else**
26 Broadcast an $Abort$ decision for T to all nodes in $\mathcal{C}_T$
27 Send back $\langle Yes, Abort\rangle$ to C^*
28 **return** $Abort$

On the participant side, the FC_{CF} protocol for each participant (Algorithm 4) is also similar to the FC_{FF} protocol (Algorithm 2), except for the following three differences. Firstly, to satisfy the liveness requirement, we apply the *transmission-before-decide* technique as proposed in [3]. Each participant will broadcast their decision to other nodes before rolling back or persisting the transaction (Algorithm 4, 22 and 26). With reliable connections, the broadcasted decision will eventually reach every non-crashed node. Therefore, when a participant is blocked for not receiving any decision within *crash timeout*, no node has decided for T. It can then decide $Abort$ independently via termination protocol (line 24). Note that if a participant votes No and decides $Abort$, we do not need to broadcast its decision, as the broadcasted No vote (line 14) has already ensured the abort of the transaction on alive participants. Secondly, each participant can quickly abort a transaction if it can detect failures in other participants (Figure 4). In particular, if the participant C_i does not receive all Yes votes from other participants within the network timeout W_{C_i}, it can directly decide $Abort$ (lines 25–28). Third, to ensure that each participant can independently decide $Abort$, we enforce the first $Commit$ decision to be made by the coordinator. A participant who collects all Yes votes will reply $\langle Yes, Undecide\rangle$ to the coordinator, and wait for a decision from either the coordinator or other participants (lines 19–24). This approach allows a participant who can not collect all Yes votes within the network timeout to decide $Abort$ independently, without concern that another participant may have already decided $Commit$.

TABLE III: Protocol comparison on ACP-introduced transaction delays (the number of message delays on participants).

Working conditions	# message delays on participants				
	2PC	3PC	PAC	EC	FC
Failure-free environment	2	4	4	2	**1**
Crash-failure env.	2	2	2	2	**1**
Network-failure env.	2	2	2	2	2

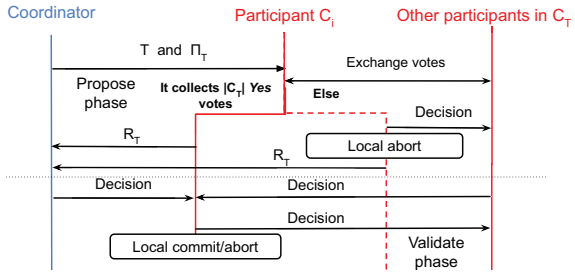


Fig. 4: Timeline of FC_{CF}. Each participant forwards their decisions to others before committing/aborting the transaction.

C. FC_{NF} for Network Failure Environment

In this environment, both crash failures and network failures may occur. There have been several ACPs proposed to handle both these two failures, such as 3PC [4], PAC [9], and Easy Commit (EC) [3]. Among them, EC is the state-of-the-art non-blocking protocol, which is well optimized for the network-failure environment. In this environment, it is challenging to design a new ACP that is better than EC. Moreover, our goal is not to design a new protocol for the specific network-failure environment but rather to design the FC protocol which can adapt well to varying failure conditions. Therefore, we directly adopt the EC protocol as the dedicated protocol FC_{NF} for the network-failure environment. By using EC, FC_{NF} can directly commit transactions without having to enter the *validate* or *propose* phases like FC_{FF} and FC_{CF}. Finally, since FC's design is modular, in the future, if there is a new ACP that can perform better than EC, we can also adopt it as FC_{NF}.

D. Protocol Comparison

We conclude this section with a comparison between existing ACPs and FC under varying failure conditions, assuming it selects the most suitable dedicated protocol. In Table III, we detail the delays imposed on distributed transactions by each ACP, quantifying how they increase transaction blocking times and intensifies the contention between concurrent transactions. Overall, FC has a minimal impact on concurrent transactions and maintains a reasonable message complexity. We observe that FC incurs additional message exchanges (votes and decisions) compared to 2PC. However, this increased message complexity does not affect its usability in OLTP workloads, as the number of participants per transaction is less than 6 in practice [3], [32] and modern systems expose

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO CEARA. Downloaded on July 29,2025 at 15:47:08 UTC from IEEE Xplore. Restrictions apply.

large network bandwidth [18], [33]. Our experiment results in Section VII-B2 and technical report [34] have further verified this point.

Table III also emphasizes our goal to design FC to adapt well to the changes in the failure condition. FC can perform well in different failure conditions. We take a transaction T involving two participants C_1, C_2 that both vote Yes as an example. In failure-free environments, FC transitions to FC_{FF}, which guarantees minimal blocking of distributed transactions. In our example, C_1 and C_2 exchange their Yes votes in 1 message delay, and both then decide $Commit$ locally with 2 Yes votes. When crash failures occur, FC switches to FC_{CF} to prevent transactions from getting blocked and quickly abort transactions affected by crash failures (1 delay). Suppose C_1 crashes, C_2 will quickly revert local changes due to not receiving 2 Yes votes in W_{C_2}. Finally, when connection delays affect FC_{FF} and FC_{CF}, FC switches to FC_{NF}. Under this condition, C_1 and C_2 are coordinated via the coordinator node without exchanging votes or decisions. Later in Section VII-C2, we will present a detailed performance evaluation to compare FC and these protocols in various environments.

V. Dedicated Protocol Selection

We now describe the protocol selector of the coordinator node (Figure 1). Recall that this component is responsible for capturing the robustness level of each participant node so that the coordinator can determine the suitable dedicated protocol (FC_{FF}, FC_{CF}, or FC_{NF}) to execute a transaction.

Definition 1 (Protocol selector)**.** *Protocol selector is a component that manages execution states of participant nodes to select the suitable protocol to execute a transaction. The execution state of each participant is modeled by a state machine called the participant state machine (PSM). Each PSM is represented by a tuple $\langle \mathcal{L}, L_0, \mathcal{E}, \delta \rangle$, where:*

- $\mathcal{L} = \{L_{\mathsf{FF}}, L_{\mathsf{CF}}, L_{\mathsf{NF}}\}$ *is a set of states, in which $L_{\mathsf{FF}}, L_{\mathsf{CF}}, L_{\mathsf{NF}}$ respectively represent the failure-free, crash-failure, and network-failure robustness levels.*
- $L_0 = L_{\mathsf{FF}}$ *is the initial state.*
- $\mathcal{E} = \{CF, NF, FF(\alpha_{\mathsf{CF}}), FF(\alpha_{\mathsf{NF}})\}$ *is a set of input events, in which CF, NF respectively indicate a crash or a network failure occurs in transaction execution, $FF(\alpha_{\mathsf{CF}})$, $FF(\alpha_{\mathsf{NF}})$ respectively represent a sequence of α_{CF} and a sequence of α_{NF} consecutive failure-free transaction executions at the crash-failure level and the network-failure level.*
- $\delta : \mathcal{L} \times \mathcal{E} \rightarrow \mathcal{L}$ *is a state transition function, which is defined as the illustration in Figure 2.*

Definition 1 formalizes the protocol selector's state machine (PSM) and its state transitions. Firstly, if protocol selector can detect a crash or a network failure during a transaction execution, it will trigger a CF or NF event to update the current state to a more stringent robustness level L_{CF} or L_{NF}. This update is called the *robustness upgrade* transition; it corresponds to the solid lines in Figure 2. Secondly, if protocol selector observes a number of failure-free consecutive executions in a high robustness-level state, it will respectively trigger the event $FF(\alpha_{\mathsf{CF}})$ or $FF(\alpha_{\mathsf{NF}})$ to move the system to the lowest robustness-level state L_{FF}. This update is called the *robustness downgrade* transition; it corresponds to the dotted lines in Figure 2. In the following subsections, we will elaborate on these failure detections and robustness transitions.

Algorithm 5: The state transition of protocol selector.

Data: L, $\mathcal{C}_T$, $\mathcal{R}_T$, α_{CF} ,α_{NF}: the current level, the set of participants, the execution results of the transaction T and two parameters of downgrade transitions.
Result: Events sent to the participants' PSMs

```
1  if L = L_FF then
2      if |R_T| < |C_T| then
3          Send CF to the PSMs of non-responsive participants.
4          Send NF to the PSMs of participants that reply late.
5      else if ⟨Yes, Undecide⟩ ∈ R_T then
6          Send NF to the PSMs of all participants in C_T.
7  else if L = L_CF then
8      if |R_T| < |C_T| then
9          Send NF to the PSMs of participants that reply late.
10     else if ⟨Yes, Abort⟩ ∈ R_T, and ⟨No, _⟩ ∉ R_T then
11         Send NF to the PSMs of all participants in C_T.
12     for C_i ∈ C_T do
13         if NumOfConsecutiveFailureFreeRuns(C_i) = α_CF then
14             Send FF(α_CF) to the PSM of C_i.
15 else                                              // L = L_NF
16     for C_i ∈ C_T do
17         if NumOfConsecutiveFailureFreeRuns(C_i) = α_NF then
18             Send FF(α_NF) to the PSM of C_i.
```

A. Robustness-Upgrade Transitions

The robustness-upgrade transition shifts PSM of a participant towards a more stringent robustness level so that subsequent transactions involving it can run with a more resilient protocol. We design FC to take a lazy approach that only upgrades the robustness level when it can detect transaction-blocking failures by analyzing past execution results. It is different from traditional approaches [35], [36] that continuously monitor the systems to predict the periods when failure can happen proactively. Such monitoring can be a new bottleneck of the system. In contrast, FC can detect failures efficiently by analyzing past execution results as follows:

First, the protocol selector can detect failures for both FC_{FF} and FC_{CF} by checking if the coordinator does not receive all participants' results, i.e., some results are missing in $\mathcal{R}_T$. For FC_{FF}, the failure type is agnostic before a crash timeout. In this case, the protocol selector first inputs CF events to the PSMs of non-responsive participants, who are likely to crash (line 3). Then, it will send NF events to protocol selector state machines of participants who reply late (line 4), since these late replies are likely caused by network delays. For FC_{CF}, it tolerates crash failures, and thus only needs to detect network failures. Like the previous case, the protocol selector will input NF events to the PSMs of participants who reply late (line 9).

Second, the protocol selector can detect network failures for both FC_{FF} and FC_{CF} by analyzing the received results. For FC_{FF}, it will report that network failures occur between participants if the coordinator receives $\langle Yes, Undecide \rangle$ (lines

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO CEARA. Downloaded on July 29,2025 at 15:47:08 UTC from IEEE Xplore. Restrictions apply.

5–6). This $Undecide$ result indicates that some votes sent to a participant did not arrive on time due to network delay. For FC_{CF}, it will conclude that a network failure occurs if there exist aborted transactions that get all Yes votes (lines 10–11). This is because if there were no network failures and all participants voted Yes, the transactions should have been committed. For both FC_{FF} and FC_{CF}, if a network failure occurred, the protocol selector cannot identify precisely the involved participants, since the failure can happen among a set of nodes. Thus, the protocol selector will shift all participants' states to the network-failure level (lines 6, 11). Conversely, crash failures are always located in individual nodes.

We prove the correctness of our failure detection rules above in our technical report (Appendix B) [34]. We also prove the soundness and completeness of the protocol selector state transition algorithm (Algorithm 5) in Appendix D.

Discussion about FC's failure detection: We notice that adopting a real-time failure detection method, such as heartbeat, could enable the faster identification of failures compared to our approach. However, we choose to detect failures based on historical transaction data mainly due to two reasons. Firstly, considering the high transaction volumes in real-world applications, the collection of information through transaction reports could be considered nearly real-time, as each transaction provides immediate feedback on the system's status. Second, our approach focuses on identifying only those failures that could affect transaction execution. It avoids the extra overhead associated with real-time failure detection techniques, such as heartbeat mechanisms, thereby enhancing system performance.

B. Robustness-Downgrade Transitions

The robustness-downgrade transitions shift the PSM state from a stringent level to a lenient one. FC triggers these transitions by monitoring the protocol execution. Specifically, if there is a participant in the crash-failure level who can consecutively execute α_{CF} transactions without any failure, then the protocol selector can transition its PSM to the failure-free level (lines 13–14). Likewise, α_{NF} is the number of consecutive failure-free transaction executions that a participant must perform at the network-failure level before its PSM can be shifted to the failure-free level (lines 17–18).

The two parameters α_{CF}, α_{NF} above control the trade-off between the protocol adaptivity and the state transition frequency. Intuitively, the smaller values of α_{CF} and α_{NF} enable FC to react faster upon the change in failure conditions, thus getting more opportunities to run a more lenient protocol. However, it also lets FC enter the costly termination protocol or coordinator-driven execution when failures recur quickly.

Generally, it is challenging to fine-tune α_{CF}, α_{NF} to optimize the performance of FC. A naive approach is to exhaustively test their possible values, but this trial-and-error method is costly. Also, this test needs to be performed every time the system is reconfigured, such as when adding a new node or replacing a machine. Building on the emerging trend of leveraging machine learning in database [37]–[39], we introduce an RL-based optimizer to automatically select the best α_{CF}, α_{NF} to mitigate the above problem. Regarding the trade-off on parameter value selection, we carefully design our RL-based optimizer to both effectively capture the degree of failure recurrence and mitigate the costs associated with protocol switching. We employ q-learning [40] as the RL learner and build our model as depicted in Figure 5.

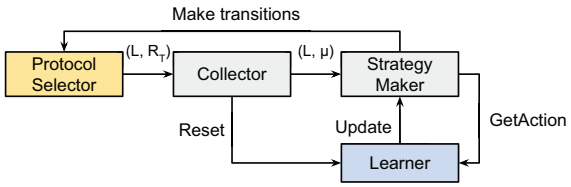


Fig. 5: Overview of the RL-based optimizer, where L, μ denote the current robustness level and the reward (average throughput).

In essence, the protocol selector asynchronously sends the execution results $\mathcal{R}_T$ with their robustness levels L to the collector. The collector resets the learner's state once it encounters a level change. Otherwise, it buffers $(L, \mathcal{R}_T)$ until enough pairs are collected. It then calculates the average throughput (transaction per second – tps) as the reward μ and sends (L, μ) to the strategy maker, who can use this information to train the learner [2]. The learner learns a Markov chain on whether to continue waiting or make the downgrade transitions regarding the current state. The strategy maker interacts with the learner during training and adjusts the participants' PSMs according to the learner's replies. After enough training, the strategy maker can cache the learned strategies as α_{CF} or α_{NF}. According to our experiment (included in our technical report [34]), the throughput gain for parameters greater than 256 is insignificant, and thus we limit the maximum value of α_{CF} and α_{NF} to 256 in our RL model. With a limited search space, the parameter tuner only takes up to 5 seconds to tune the parameters. The users can call the RL-based optimizer after a time interval (e.g., hourly) or when the system is reconfigured (e.g. replacing a node).

Our adoption of the RL-based optimizer inherently relies on an assumption that failures can be predicted based on historical data, which has been adopted in existing research [16], [17]. A more detailed analysis of this assumption can be found in our technical report [34], in which we analyze failure traces extracted from 10 real-world applications [12], and observe the predictability of current failure probability via history collected in the previous hour.

VI. Failure Handling and Correctness of FC

A. Failure handling

The failure handling of ACP includes the termination of blocked transactions that experience *crash timeout*, and the recovery of half-executed transactions on recovered nodes. We described the use of termination protocol in FC_{CF}, FC_{FF} in

[2]Users can decide which reward function is suitable for their applications. In this work, we use the average throughput. Other systems [26], [41], [42] value tail latency, while [43] uses throughput, and [44], [45] utilize both and thus express their requirements using more complex functions.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO CEARA. Downloaded on July 29,2025 at 15:47:08 UTC from IEEE Xplore. Restrictions apply.

Sections IV-A, IV-B. For crash recovery, FC adds transaction logs into non-volatile storage and recovers from crash failures like existing ACPs [2], [3]. We now elaborate distinctive approaches of FC_{FF} and FC_{CF} in handling failures compared to the commonly used 2PC. For more details, please refer to our full explanation in our technical report [34] (Appendix A, C).

Compared with 2PC, FC_{FF} requires longer blocking in its failure recovery and termination. A node needs to either receive a decision from others, collect all Yes votes, or receive a No vote to decide locally. This is because, unlike 2PC, FC_{FF} does not enforce the first $Commit$ decision to be made on the coordinator. As a result, a recovered/terminated node may not be able to directly decide by checking coordinator logs. For example, suppose FC_{FF} is picked for a transaction involving the two participants C_1 and C_2 and the coordinator C^*. If C_1 votes Yes but C_2 crashes before sending any vote, C_1 and C^* will be blocked to wait for the recovery of C_1, as they do not know whether C_2 decided $Abort$ due to local No vote (Algorithm 2 lines 13-15) or decided $Commit$ with 2 Yes votes (Algorithm 2 lines 20-22).

In contrast, FC_{CF} requires shorter blocking than 2PC in its failure handling. As discussed in Section IV-B, FC_{CF} supports the independent termination of each node that experiences *crash timeout*. We note that the design of non-blocking ACP with the unbounded network has been proven impossible [21], [46]. If FC_{CF} must be executed in settings with infinite *crash timeout*, it obtains the same level of liveness as 2PC while still benefiting from the shorter delay of distributed transactions (Table III).

B. Correctness

As discussed in Section III, FC selects a single protocol for each transaction execution, and therefore, there is no case where different dedicated protocols are used for a single transaction. The correctness of each transaction can thus be guaranteed exclusively by the selected protocol itself. Next, we prove the correctness of FC by presenting the correctness proofs of its dedicated protocols FC_{FF}, FC_{CF}, and FC_{NF}. FC_{NF} is identical to the EasyCommit, and its correctness is proved in [3]. In the following, we will briefly describe the correctness proofs of FC_{FF} and FC_{CF}. Due to space constraints, more detailed correctness proof can be found in our technical report [34]. In our technical report, we have further verified the correctness of these dedicated protocols using TLA+ specification [47] and attached a formal proof of overall FC, which details all failure cases.

In FC_{FF}, a node can only decide for a transaction T in three ways: (i) commit with $|\mathcal{C}_T|$ Yes votes (Algorithm 1 lines 7-9, Algorithm 2 lines 20-22 and 26, Section VI-A), (ii) abort with No vote (Algorithm 2 lines 13-15 17-19 and 26, Section VI-A), (iii) or decide following others (Algorithm 1 lines 4-6, Algorithm 2 lines 24-25 and 26, Section VI-A). The nodes which follow cases (i) and (ii) to decide by themselves always ensure the safety property (Section II-A), since each node only votes once (*agreement*) and a node deciding $Commit$ must have collected all Yes votes (*commit validity*). The safety is also guaranteed on other nodes who decide to follow peers, as the copy of the decision does not break *agreement* property. Meanwhile, the existence of a self-deciding node has already guaranteed *commit validity*.

FC_{CF} ensures safety property by enforcing the coordinator to be the first node that decides $Commit$ (Algorithm 3 lines 7-9). Such $Commit$ decision is only made when all nodes vote Yes, which ensures *commit validity* property. For *agreement* property, FC_{CF} blocks all participants at $Undecide$ state (Algorithm 4 lines 19-24) during the above progress. The *transmission-before-decide* technique (Section IV-B) and coordinator log checking (Section VI-A) then ensure all alive or recovered nodes decide $Commit$ accordingly.

VII. Evaluation

We have implemented FC in a prototype system using mostly Golang, except the reinforcement learning component is written in Python. Our system processes transactions similarly to Google's Percolator [18]. Each transaction retries when it gets aborted due to conflicts (at most 10 times). Our system does not retry transactions that cannot commit due to consistency requirements or system failures. We have also implemented FC on two popular real-world database systems: MongoDB, representing the NoSQL databases, and PostgreSQL, representing relational databases. For more implementation details and codes, please refer to our codebase at: https://github.com/allvphx/Flexi.

We have conducted extensive experiments to evaluate and compare the performance of FC with state-of-the-art ACPs: 2PC [2], 3PC [4], Easy Commit (EC) [3], and PAC [9]. For the baselines 2PC, 3PC, and EC, we adopted the implementation design in [3]. For PAC [9], we implemented its centralized variant, denoted as C-PAC, where the coordinator is fixed as the leader. Note that, in the main paper, we do not compare FC with protocols designed for systems that replicate data across different nodes, such as TAPIR [8], G-PAC [9], since their settings are different from FC's system model. In the following, we will present our detailed experimental results regarding two main aspects: (1) performance comparison of FC versus other protocols and (2) impact of FC's optimizations (FC_{FF} and FC_{CF}, RL-tuned α_{CF}, α_{NF}) on its performance. Due to the page limit, we have moved the details of less critical experiments, such as the impact of network delays and sensitivity analysis, to our technical report [34].

A. Experiment Setup

We conducted experiments in two different network settings: (i) a local data cluster and (ii) a cross-region cloud environment spanning 3 continents. In the first setting, we utilize 11 server nodes deployed in a data cluster with Ubuntu 20.04 system. Each server consists of two 24-core 3.0GHz Intel(R) Xeon(R) Gold 5317 CPU processors, 128GB of DRAM, and connects to the cluster by 1Gbps network with 0.3ms network delay. Here, we simulate a cross-datacenter environment by adding 10ms latency to packages sent among these nodes.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO CEARA. Downloaded on July 29,2025 at 15:47:08 UTC from IEEE Xplore. Restrictions apply.

In the second setting of a real cross-region data center, we use Google Cloud Platform with four Google E2-medium machines distributed across four different regions: Singapore, Sydney, Toronto, and Tokyo. In all experiments, we configure one node as the coordinator while the others are participants. Each node maintains a single server process that handles requests from others using lightweight execution threads. On the coordinator, we use ACPs for cross-shard transactions and commit all single-shard transactions in a single message round trip as in [3], [48], [49].

For micro benchmarking, we follow the existing works to run the experiment on the YCSB benchmark [9], [15], [50], [51]. Each client continuously generates read and write multi-record transactions with closed loops, whose data access distribution fits Zipfian distribution [50] with a skew factor that controls contention. We make 20% transactions in this benchmark cross-shard by default. Tail latency (99% latency or P99 latency) is used to measure transaction latency since it can reflect precisely user experiences. This latency is a widely measured metric when evaluating databases [41], [52]–[54].

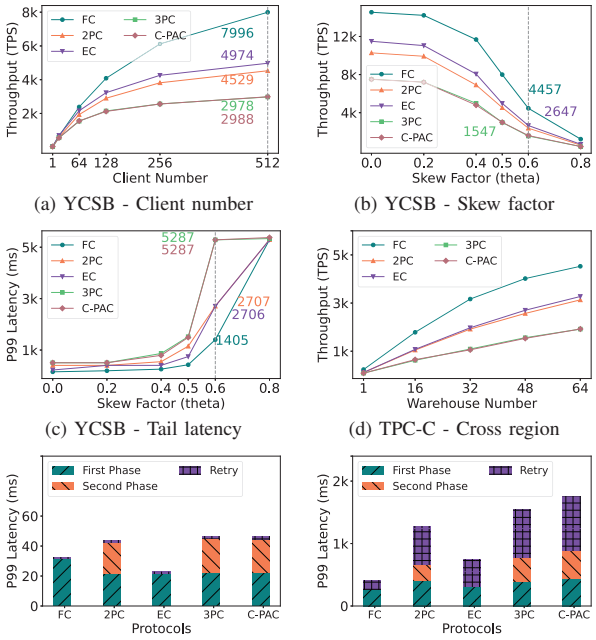
For macro benchmarking, we use TPC-C, a standard benchmark for OLTP systems that include 3 types of read-write and 2 types of read-only transactions [32]. We utilize up to 64 warehouses located in every server node. Each warehouse contains 10 different districts, and each district maintains the information of 3000 customers. We use closed loops to simulate clients that keep sending transactions to the coordinator and control the contention by adjusting the warehouse number.

We conduct the experiments in two operating environments: (i) the failure-free environment, and (ii) the failure environment where crash and network failures frequently occur. To simulate real-world failure environments, we inject failures to nodes and network connections following Poisson/Truncated normal random process [55], [56] and let each failure last for 1 second. A parameter τ controls the expected time for a failure to recur after recovery. By default, we set the number of clients to 512 for the YCSB benchmark and 2000 for the TPC-C benchmark. Transaction length is configured to 16, skew factor to 0.5, and network buffer parameter r to 2. We also follow the existing works [9], [57] to set the number of participant nodes to 3 when assessing the commit protocols' detailed performance. An experiment with varying numbers of participants, ranging from 3 to 10, is also conducted to evaluate their scalability.

B. Performance in Failure-Free Environment

In this environment, no failure will be injected. We start to record the experimental data 5 seconds after all the machines have warmed up. All tests are run 5 times and we report their average results.

1) Experiment with the YCSB and TPC-C benchmarks, under varying varying settings: Figure 6 presents our experimental results, in which, FC demonstrates a significant gain in throughput, with a substantial reduction in latency in all these settings and benchmarks. For example, when the skew factor is 0.6 (Figures 6b and 6c), it achieves from 1.68x to 2.88x

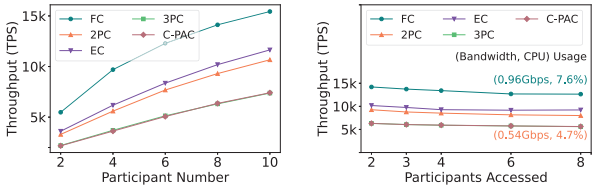


(a) YCSB - Client number (b) YCSB - Skew factor
(c) YCSB - Tail latency (d) TPC-C - Cross region
(e) Breakdown - 16 clients (f) Breakdown - 512 clients

Fig. 6: Performance of all protocols with varying contentions.

throughput speedup and from 48.1% to 73.4% latency decrease compared to other protocols. These better performances of FC are contributed by the dedicated protocol FC_{FF} in this failure-free environment. In particular, FC_{FF} requires only 1 message delay on each participant to commit or abort transactions, while all other protocols EC, 2PC, 3PC, and C-PAC require at least 2 message delays (Table III). Figure 6e and 6f present the latency breakdown of all protocols. In the low contention setting, all protocols exhibit similar low latency. However, as contention goes high, FC achieves much better performance compared to other protocols, due to its shorter transaction blocking window. This further verifies our idea to reduce ACP-introduced transaction blocking for better performance.

We have also conducted experiments to study the performance of FC in the cross-region setting across Singapore, Sydney, Toronto, and Tokyo, as described in Section VII-A. Figure 6d shows that FC still demonstrates a notable speedup compared to other protocols. This result validates the resilience of FC in the face of wide area network variability.



(a) Scalability (b) Varying # of accessed participants

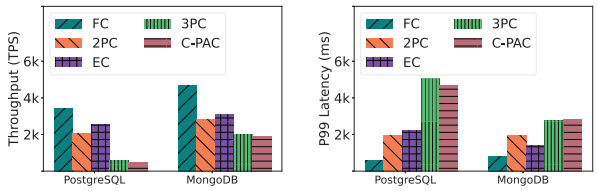
Fig. 7: Protocol performance with varying participant number($|S|$), participants accessed per transaction ($|\mathcal{C}_T|$) under YCSB benchmark.

2) Scalability evaluation of all protocols with the YCSB benchmark, under a varying number of total participants or

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO CEARA. Downloaded on July 29,2025 at 15:47:08 UTC from IEEE Xplore. Restrictions apply.

accessed participants: Figure 7 studies how FC and other protocols can scale with a larger number of participants and a larger number of participants accessed by each transaction. As demonstrated in Figure 7a, all protocols exhibit nearly linear scalability as the number of participants increases. Furthermore, despite varying server counts and accessed participant numbers, FC consistently outperforms other protocols. Among all settings in Figure 7, FC obtains from 1.33x to 2.52x throughput speedup than other protocols. This performance gain is due to the same reason explained previously: the protocol FC_{FF}, which operates in this failure-free environment, requires fewer transaction blocking than other protocols.

In Figure 7b, we measure the peak resource utilization of 2PC and FC when transactions involve 8 participants to study the impact of FC's higher message complexity. As reported, FC only requires an additional bandwidth overhead of 0.42 Gbps (less than 1% of the maximum bandwidth) and an increase in CPU usage by 2.9%, primarily attributed to its increase in throughput. Such findings further verify our assertion in Section IV-D that FC's increased message complexity does not affect its usage in OLTP workloads.



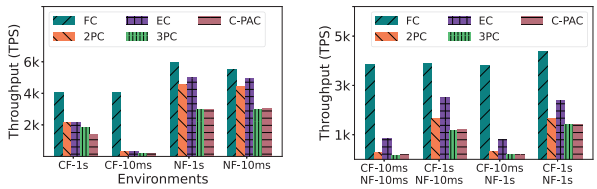
(a) Throughput (b) Tail latency

Fig. 8: Performance of all protocols on PostgreSQL and MongoDB under YCSB benchmark.

3) Evaluation on real-world databases: One major contribution of FC is that it can significantly reduce the participant-side blocking window, effectively mitigating transaction contention. However, this transaction blocking reduction may vary across different databases, due to differences in their concurrency control, indexes, data format, and system designs. To study the effect of FC on real-world databases, we implemented it on two prominent database systems: PostgreSQL and MongoDB. For integration, we have developed system-specific extensions for each database. In PostgreSQL, we leverage its built-in transaction support to implement all ACPs. For MongoDB, since it does not support native serializable multi-document transactions, we have implemented an in-memory per-document lock manager that works with MongoDB to support serializable transactions. During execution, each transaction first obtains shared/exclusive locks for the documents it accesses from the lock manager. With the acquired latches, it can then safely call the *findOne/updateOne* APIs to access documents in MongoDB. We let each transaction access 4 rows/documents and set the skew factor to 0.8. To reduce the cost associated with maintaining connections, we also limited the number of clients in PostgreSQL to 128. The results, as illustrated in Figure 8, demonstrate the significance of FC in real-world database systems. Compared to other ACPs, FC exposes remarkable improvements in both throughput and latency, achieving a speedup ranging from 1.3x to 7.2x in throughput, while reducing tail latency by 44% to 88%.

C. Performance on Failure Environment

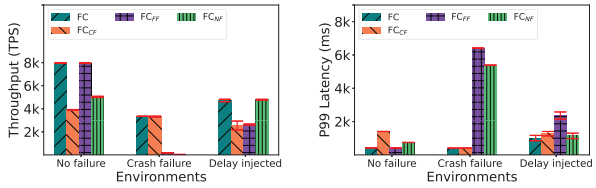
In this second part of our evaluation, we benchmark FC and other protocols on environments where crash failures and network failures can occur. We simulate these failures by frequently injecting crashes or delays into nodes and network connections [3]. This injection is controlled by the parameter τ (Section VII-A): the expected time for a failure to recur after recovery. By default, we employ a Poisson random process to model the recurrence of failures [55].



(a) Either crash or network failures (b) Both crash and network failures

Fig. 9: Performance of all protocols with environments having failures with varying failure recurrence time under YCSB benchmark.

1) Experiment with the YCSB benchmark and environments containing failures: Figure 9 presents the experimental results in different environments where failures happen on a participant. We run each test for 100 seconds to accommodate long failure recurrence periods. Here, the notation CF-τ/NF-τ means that the crash failure/network failure is configured to recur after τ time. Figures 9a and 9b show that in all failure settings, FC always achieves better performance than other protocols. In particular, it obtains from 1.1x to 13.5x throughput speedup, compared to the next two best protocols EC and 2PC. The improvement is derived from FC's capability to select and execute the most suitable protocol among FC_{FF}, FC_{CF}, and FC_{NF}: while FC_{FF} reduces contention between transactions in failure-free environments, FC_{CF} and FC_{NF} mitigate the impact of crash and network failures on committable transactions, respectively. This combination ensures efficient transaction processing under changing environments.



(a) Throughput (b) Tail Latency

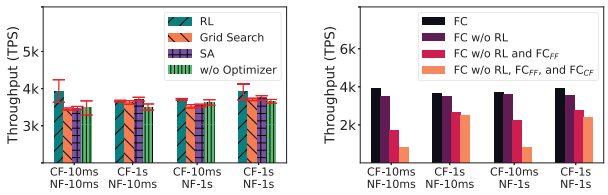
Fig. 10: Performance comparison of FC and its dedicated protocols with varying environments under YCSB benchmark

2) Effect of dedicated protocols: We now study the effect of FC dedicated protocols by evaluating their performance in dif-

[3]For crash failures, we pick one participant and stop it. For network failures, we delay messages sent between different nodes following a truncated normal distribution [56].

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO CEARA. Downloaded on July 29,2025 at 15:47:08 UTC from IEEE Xplore. Restrictions apply.

ferent environments: failure-free, crash-failure, and network-failure environments. For crash failures, we stopped one participant and set $|\mathcal{C}_T|$ to 3, making all distributed transactions access this node. For network failures, we inject delays into all messages sent between participants. As shown in Figure 10, each dedicated protocol performs the best under its expected environment. For instance, in the failure-free environment, FC_{FF} achieves 2.03x and 1.58x speed up in throughput while reducing the latency by 70.3% and 43.6%, compared to FC_{CF} and FC_{NF} respectively. More importantly, across all experimental settings, the combined protocol FC consistently achieves comparable or superior performance compared to dedicated protocols. This result verifies FC's ability to switch to a suitable protocol regarding varying failure conditions.



(a) RL vs. other methods (b) Ablation study

Fig. 11: Effects of RL on FC under YCSB benchmark.

3) Effects of reinforcement learning (RL): Figure 11a represents a comparison of FC when using different parameter tuners. Our results reveal that FC achieves better performance when integrating RL as its parameter tuner, compared to two heuristic methods, Simulated Annealing (SA) and Grid Search, and a setting without any optimizer. We attribute it to RL's capability to dynamically adapt its strategy based on immediate feedback, allowing for optimizations before the completion of predefined α_{CF}/α_{NF} steps. In contrast, both SA and Grid Search start their parameter adjustments only after the entire α_{CF}/α_{NF} run cycle, relying on aggregated throughput to inform their optimizations.

We have also validated the contributions of FC's optimizations through an ablation study. As shown in Figure 11b, the adoption of the RL-based optimizer, alongside the dedicated protocols FC_{FF} and FC_{CF}, all notably improve overall system performance.

VIII. Related Works

Atomic Commit Protocol (ACP) for distributed transactions is an important topic in database systems. Initially proposed by Skeen in [23], ACP sees a series of improvements in [24], [58], [59]. Among them, 2PC [2] is widely recognized as the golden standard in the industry but is also well-known to be the bottleneck of distributed systems for its low efficiency. Existing ACPs optimize 2PC by (i) enhancing the performance during normal processing, (ii) improving availability by ensuring liveness property, or (iii) developing adaptive protocols that adjust based on the workload characteristics.

To enhance the performance during normal processing, PrA and PrC were introduced in [60], which use additional logs to reduce the cost of aborting or committing messages. PrC was further optimized in [61] by giving up knowledge about the transactions prior to coordinator failures to enhance its performance. Nonetheless, these protocols and their variants in [61]–[63] cannot reduce the participant-side message delay. To improve such participant-side message delay, 1PC [64] and its variants [65]–[67] sacrifice the generalizability by adding additional strong assumptions, while O2PC [68] compromises the atomicity when a transaction fails. Compared with them, FC can execute transactions more efficiently without compromising the generalizability or the ACID properties.

Several works have also been proposed to improve the availability of ACPs. 3PC and its variants [3], [4], [27] address 2PC's lack of termination when nodes fail by adding extra messages or communication steps. Paxos Commit and its variants [6], [8], [9], [69] ensure the liveness of 2PC under node failures by replication. Unlike them, FC does not require replicas or extra message round trips in normal processing. Moreover, while other ACPs assume a fixed failure condition, FC monitors the current failure condition and dynamically switches to a suitable dedicated protocol for higher efficiency.

There are also other ACPs that can switch to operate a suitable protocol when transaction behavior changes. For example, ACOM [70] shifts between PrA and PrC according to transaction behavior on each node. ML1-2PC [22], [71] can dynamically select either 1PC or 2PC based on transaction behavior and system requirements. While existing adaptive ACPs all adjust protocol based on workload characteristics, FC is the only protocol that can adjust itself to a suitable dedicated protocol according to failure conditions.

IX. Conclusions

We have presented FC, a robust failure-aware atomic commit protocol for distributed transactions. FC includes a novel state transition model protocol selector to infer the failure conditions. Reinforcement learning is also applied to boost its adaptivity and performance in unstable environments. We have implemented a distributed transactional key-value storage with FC and evaluated its performance. The results show that FC achieves up to 2.88x improvement in throughput and up to 3.76x speedup in latency, compared to existing protocols 2PC, 3PC, Easy Commit, and C-PAC. We have also evaluated the effectiveness of reinforcement learning on tuning FC's parameters. The results indicate that the tuning can improve the performance of FC in unstable environments.

Acknowledgements

We sincerely thank the reviewers for their valuable feedback. The research is supported by the National Research Foundation, Singapore under its Emerging Areas Research Projects (EARP) Funding Initiative. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore. Meihui Zhang's work is supported by National Natural Science Foundation of China (62072033).

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO CEARA. Downloaded on July 29,2025 at 15:47:08 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] J. Gray and A. Reuter, *Transaction processing: concepts and techniques*. Elsevier, 1992.

[2] N. Nouali, A. Doucet, and H. Drias, "A two-phase commit protocol for mobile wireless environment," in *Proceedings of the 16th Australasian database conference-Volume 39*, 2005, pp. 135–143.

[3] S. Gupta and M. Sadoghi, "Easycommit: A non-blocking two-phase commit protocol." in *EDBT*, 2018, pp. 157–168.

[4] N. Kumar, L. Sahoo, and A. Kumar, "Design and implementation of three phase commit protocol (3pc) algorithm," in *2014 International Conference on Reliability Optimization and Information Technology*, 2014, pp. 116–120.

[5] S. Gupta and M. Sadoghi, "Efficient and non-blocking agreement protocols," *Distributed and Parallel Databases*, vol. 38, no. 2, pp. 287–333, 2020.

[6] J. Gray and L. Lamport, "Consensus on transaction commit," *ACM Transactions on Database Systems (TODS)*, vol. 31, no. 1, pp. 133–160, 2006.

[7] T. Kraska, G. Pang, M. J. Franklin, S. Madden, and A. D. Fekete, "MDCC: multi-data center consistency," in *EuroSys*. ACM, 2013, pp. 113–126.

[8] I. Zhang, N. K. Sharma, A. Szekeres, A. Krishnamurthy, and D. R. Ports, "Building consistent transactions with inconsistent replication," *ACM Transactions on Computer Systems*, vol. 35, no. 4, pp. 1–37, 2018.

[9] S. Maiyya, F. Nawab, D. Agrawal, and A. E. Abbadi, "Unifying consensus and atomic commitment for effective cloud data management," *Proceedings of the VLDB Endowment*, vol. 12, no. 5, pp. 611–623, 2019.

[10] T. Benson, S. Sahu, A. Akella, and A. Shaikh, "A first look at problems in the cloud," in *2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 10)*, 2010.

[11] X. Li, G. Yu, P. Chen, H. Chen, and Z. Chen, "Going through the life cycle of faults in clouds: Guidelines on fault handling," in *2022 IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2022, pp. 121–132.

[12] S. Talluri, L. Overweel, L. Versluis, A. Trivedi, and A. Iosup, "Empirical characterization of user reports about cloud failures," in *2021 IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS)*. IEEE, 2021, pp. 158–163.

[13] X. Yu, Y. Xia, A. Pavlo, D. Sánchez, L. Rudolph, and S. Devadas, "Sundial: Harmonizing concurrency control and caching in a distributed OLTP database management system," *Proc. VLDB Endow.*, vol. 11, no. 10, pp. 1289–1302, 2018. [Online]. Available: http://www.vldb.org/pvldb/vol11/p1289-yu.pdf

[14] Z. Guo, X. Zeng, K. Wu, W. Hwang, Z. Ren, X. Yu, M. Balakrishnan, and P. A. Bernstein, "Cornus: Atomic commit for a cloud DBMS with storage disaggregation," *Proc. VLDB Endow.*, vol. 16, no. 2, pp. 379–392, 2022. [Online]. Available: https://www.vldb.org/pvldb/vol16/p379-guo.pdf

[15] R. Harding, D. V. Aken, A. Pavlo, and M. Stonebraker, "An evaluation of distributed concurrency control," *Proc. VLDB Endow.*, vol. 10, no. 5, pp. 553–564, 2017. [Online]. Available: http://www.vldb.org/pvldb/vol10/p553-harding.pdf

[16] Y. Tan and X. Gu, "On predictability of system anomalies in real world," in *2010 IEEE International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems*. IEEE, 2010, pp. 133–140.

[17] Y. Xu, K. Sui, R. Yao, H. Zhang, Q. Lin, Y. Dang, P. Li, K. Jiang, W. Zhang, J.-G. Lou *et al.*, "Improving service availability of cloud systems by predicting disk error," in *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, 2018, pp. 481–494.

[18] D. Peng and F. Dabek, "Large-scale incremental processing using distributed transactions and notifications," *12th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 16)*, 2010.

[19] J. Gray and L. Lamport, "Consensus on transaction commit," *ACM Transactions on Database Systems (TODS)*, vol. 31, no. 1, pp. 133–160, 2006.

[20] R. A. DeMillo, N. A. Lynch, and M. J. Merritt, "Cryptographic protocols," in *Proceedings of the fourteenth annual ACM symposium on Theory of computing*, 1982, pp. 383–400.

[21] M. J. Fischer, N. A. Lynch, and M. S. Paterson, "Impossibility of distributed consensus with one faulty process," *Journal of the ACM (JACM)*, vol. 32, no. 2, pp. 374–382, 1985.

[22] Y. J. Al-Houmaily and P. K. Chrysanthis, "1-2pc: The one-two phase atomic commit protocol," in *Proceedings of the 2004 ACM Symposium on Applied Computing*, ser. SAC '04. New York, NY, USA: Association for Computing Machinery, 2004, p. 684–691.

[23] D. Skeen, "Nonblocking commit protocols," in *Proceedings of the 1981 ACM SIGMOD International Conference on Management of Data, Ann Arbor, Michigan, USA, April 29 - May 1, 1981*, Y. E. Lien, Ed. ACM Press, 1981, pp. 133–142. [Online]. Available: https://doi.org/10.1145/582318.582339

[24] R. Guerraoui and J. Wang, "How fast can a distributed transaction commit?" in *Proceedings of the 36th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*, ser. PODS '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 107–122.

[25] T. D. Chandra, "Unreliable failure detectors for asynchronous distributed systems," Ph.D. dissertation, Cornell University, USA, 1993.

[26] X. Yan, L. Yang, and B. Wong, "Domino: using network measurements to reduce state machine replication latency in wans," in *Proceedings of the 16th International Conference on emerging Networking EXperiments and Technologies*, 2020, pp. 351–363.

[27] D. Skeen, "A quorum-based commit protocol," Cornell University, Tech. Rep., 1982.

[28] C. G. Gray and D. R. Cheriton, "Leases: An efficient fault-tolerant mechanism for distributed file cache consistency," in *Proceedings of the Twelfth ACM Symposium on Operating System Principles, SOSP 1989, The Wigwam, Litchfield Park, Arizona, USA, December 3-6, 1989*, G. R. Andrews, Ed. ACM, 1989, pp. 202–210. [Online]. Available: https://doi.org/10.1145/74850.74870

[29] L. Yang, X. Yan, and B. Wong, "Natto: Providing distributed transaction prioritization for high-contention workloads," in *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022*, Z. G. Ives, A. Bonifati, and A. E. Abbadi, Eds. ACM, 2022, pp. 715–729. [Online]. Available: https://doi.org/10.1145/3514221.3526161

[30] J. Geng, A. Sivaraman, B. Prabhakar, and M. Rosenblum, "Nezha: Deployable and high-performance consensus using synchronized clocks," *Proc. VLDB Endow.*, vol. 16, no. 4, pp. 629–642, 2022. [Online]. Available: https://www.vldb.org/pvldb/vol16/p629-geng.pdf

[31] B. Liskov, "Practical uses of synchronized clocks in distributed systems," *Distributed Comput.*, vol. 6, no. 4, pp. 211–219, 1993. [Online]. Available: https://doi.org/10.1007/BF02242709

[32] T. P. P. Council, "Tpc benchmark c (standard specification, revision 5.11), 2010," *URL: http://www. tpc. org/tpcc*, p. 100, 2010.

[33] "Amazon aws," https://aws.amazon.com/, 2022, online; accessed: 2022-12-11.

[34] (2023) Fc: Adaptive atomic commit via failure detection (extended version). [Online]. Available: https://drive.google.com/file/d/1GnpUufv1loDuzl5d_zm-842mlM2TxgYp/view

[35] L. Yu, Z. Zheng, Z. Lan, and S. Coghlan, "Practical online failure prediction for blue gene/p: Period-based vs event-driven," in *2011 IEEE/IFIP 41st International Conference on Dependable Systems and Networks Workshops (DSN-W)*. IEEE, 2011, pp. 259–264.

[36] Y. Zhang and A. Sivasubramaniam, "Failure prediction in IBM bluegene/l event logs," in *22nd IEEE International Symposium on Parallel and Distributed Processing, IPDPS 2008, Miami, Florida USA, April 14-18, 2008*. IEEE, 2008, pp. 1–5. [Online]. Available: https://doi.org/10.1109/IPDPS.2008.4536397

[37] W. Wang, M. Zhang, G. Chen, H. V. Jagadish, B. C. Ooi, and K. Tan, "Database meets deep learning: Challenges and opportunities," *SIGMOD Rec.*, vol. 45, no. 2, pp. 17–22, 2016. [Online]. Available: https://doi.org/10.1145/3003665.3003669

[38] B. C. Ooi, K. Tan, S. Wang, W. Wang, Q. Cai, G. Chen, J. Gao, Z. Luo, A. K. H. Tung, Y. Wang, Z. Xie, M. Zhang, and K. Zheng, "SINGA: A distributed deep learning platform," in *Proceedings of the 23rd Annual ACM Conference on Multimedia Conference, MM '15, Brisbane, Australia, October 26 - 30, 2015*, X. Zhou, A. F. Smeaton, Q. Tian, D. C. A. Bulterman, H. T. Shen, K. Mayer-Patel, and S. Yan, Eds. ACM, 2015, pp. 685–688. [Online]. Available: https://doi.org/10.1145/2733373.2807410

[39] G. Li, X. Zhou, and L. Cao, "AI meets database: AI4DB and DB4AI," in *SIGMOD '21: International Conference on Management of Data, Virtual Event, China, June 20-25, 2021*, G. Li, Z. Li, S. Idreos, and D. Srivastava, Eds. ACM, 2021, pp. 2859–2866. [Online]. Available: https://doi.org/10.1145/3448016.3457542

[40] C. J. Watkins and P. Dayan, "Q-learning," *Machine learning*, vol. 8, no. 3-4, pp. 279–292, 1992.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO CEARA. Downloaded on July 29,2025 at 15:47:08 UTC from IEEE Xplore. Restrictions apply.

[41] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels, "Dynamo: Amazon's highly available key-value store," *ACM SIGOPS operating systems review*, vol. 41, no. 6, pp. 205–220, 2007.

[42] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild *et al.*, "Spanner: Google's globally distributed database," *ACM Transactions on Computer Systems*, vol. 31, no. 3, pp. 1–22, 2013.

[43] B. Debnath, S. Sengupta, and J. Li, "Flashstore: High throughput persistent key-value store," *Proceedings of the VLDB Endowment*, vol. 3, no. 1-2, pp. 1414–1425, 2010.

[44] G. Li, X. Zhou, S. Li, and B. Gao, "Qtune: A query-aware database tuning system with deep reinforcement learning," *Proceedings of the VLDB Endowment*, vol. 12, no. 12, pp. 2118–2130, 2019.

[45] H. Yoon, J. Yang, S. F. Kristjansson, S. E. Sigurdarson, Y. Vigfusson, and A. Gavrilovska, "Mutant: Balancing storage cost and latency in lsm-tree data stores," in *Proceedings of the ACM Symposium on Cloud Computing*, 2018, pp. 162–173.

[46] R. Guerraoui, M. Larrea, and A. Schiper, "Non-blocking atomic commitment with an unreliable failure detector," in *14th Symposium on Reliable Distributed Systems, SRDS 1995, Bad Neuenahr, Germany, September 13-15, 1995, Proceedings*. IEEE Computer Society, 1995, pp. 41–50. [Online]. Available: https://doi.org/10.1109/RELDIS.1995.518722

[47] L. Lamport, *Specifying Systems, The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. [Online]. Available: http://research.microsoft.com/users/lamport/tla/book.html

[48] Y. Lu, X. Yu, L. Cao, and S. Madden, "Epoch-based commit and replication in distributed oltp databases," *Proc. VLDB Endow.*, 2021.

[49] R. Harding, D. Van Aken, A. Pavlo, and M. Stonebraker, "An evaluation of distributed concurrency control," *Proc. VLDB Endow.*, vol. 10, no. 5, p. 553–564, jan 2017.

[50] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with ycsb," in *Proceedings of the 1st ACM symposium on Cloud computing*, 2010, pp. 143–154.

[51] T. M. Qadah and M. Sadoghi, "Highly available queue-oriented speculative transaction processing," *arXiv:2107.11378*, 2021.

[52] X. Chen, H. Song, J. Jiang, C. Ruan, C. Li, S. Wang, G. Zhang, R. Cheng, and H. Cui, "Achieving low tail-latency and high scalability for serializable transactions in edge computing," in *Proceedings of the Sixteenth European Conference on Computer Systems*, 2021, pp. 210–227.

[53] L. Suresh, M. Canini, S. Schmid, and A. Feldmann, "C3: Cutting tail latency in cloud data stores via adaptive replica selection," in *12th USENIX Symposium on Networked Systems Design and Implementation*, 2015, pp. 513–527.

[54] G. Prekas, M. Kogias, and E. Bugnion, "Zygos: Achieving low tail latency for microsecond-scale networked tasks," in *Proceedings of the 26th Symposium on Operating Systems Principles*, 2017, pp. 325–341.

[55] W. Barreto-Souza and F. Cribari-Neto, "A generalization of the exponential-poisson distribution," *Statistics & Probability Letters*, vol. 79, no. 24, pp. 2493–2500, 2009.

[56] T. Elteto and S. Molnar, "On the distribution of round-trip delays in tcp/ip networks," in *Proceedings 24th Conference on Local Computer Networks. LCN'99*. IEEE, 1999, pp. 172–181.

[57] M. Abdallah, R. Guerraoui, and P. Pucheral, "One-phase commit: does it make sense?" in *Proceedings 1998 International Conference on Parallel and Distributed Systems (Cat. No.98TB100250)*, 1998, pp. 182–192.

[58] R. Guerraoui, "Non-blocking atomic commit in asynchronous distributed systems with failure detectors," *Distributed Computing*, vol. 15, no. 1, pp. 17–25, 2002.

[59] R. Guerraoui, V. Hadzilacos, P. Kuznetsov, and S. Toueg, "The weakest failure detectors to solve quittable consensus and nonblocking atomic commit," *SIAM Journal on Computing*, vol. 41, 01 2012.

[60] C. Mohan, B. Lindsay, and R. Obermarck, "Transaction management in the r* distributed database management system," *ACM Transactions on Database Systems (TODS)*, vol. 11, no. 4, pp. 378–396, 1986.

[61] B. Lampson and D. Lomet, "A new presumed commit optimization for two phase commit," in *19th International Conference on Very Large Data Bases (VLDB'93)*, 1993, pp. 630–640.

[62] Y. J. Al-Houmaily, P. K. Chrysanthis, and S. P. Levitan, "Enhancing the performance of presumed commit protocol," in *Proceedings of the 1997 ACM symposium on Applied computing*, 1997, pp. 131–133.

[63] P. Chrysanthis, G. Samaras, and Y. Al-Houmaily, "Recovery and performance of atomic commit processing in distributed database systems," *Recovery Mechanisms in Database Systems*, pp. 370–416, 1998.

[64] Y. Al-houmaily and P. Chrysanthis, "The implicit-yes vote commit protocol with delegation of commitment," in *Proc. of 9th Intl. Conf. on Parallel and Distributed Computing Systems*, 09 1996.

[65] J. W. Stamos and F. Cristian, "A low-cost atomic commit protocol," in *Proceedings Ninth Symposium on Reliable Distributed Systems*. IEEE, 1990, pp. 66–75.

[66] M. Stonebraker, "Concurrency control and consistency of multiple copies of data in distributed ingres," *IEEE Transactions on software Engineering*, no. 3, pp. 188–194, 1979.

[67] J. W. Stamos and F. Cristian, "Coordinator log transaction execution protocol," *Distributed and Parallel Databases*, vol. 1, no. 4, pp. 383–408, 1993.

[68] E. Levy, H. F. Korth, and A. Silberschatz, "An optimistic commit protocol for distributed transaction management," *ACM SIGMOD Record*, vol. 20, no. 2, pp. 88–97, 1991.

[69] M. Hammer and D. Shipman, "Reliability mechanisms for sdd-1: A system for distributed databases," *ACM Transactions on Database Systems*, vol. 5, no. 4, pp. 431–466, 1980.

[70] P. Serrano-Alvarado, R. Rouvoy, and P. Merle, "Self-adaptive component-based transaction commit management," in *Proceedings of the 4th Workshop on Reflective and Adaptive Middleware Systems*, ser. ARM '05. New York, NY, USA: Association for Computing Machinery, 2005.

[71] Y. Al-houmaily and P. Chrysanthis, "Ml-1-2pc: An adaptive multi-level atomic commit protocol," in *East European Conference on Advances in Databases and Information Systems*, vol. 3255, 09 2004, pp. 275–290.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO CEARA. Downloaded on July 29,2025 at 15:47:08 UTC from IEEE Xplore. Restrictions apply.